Grafos e Caminho Mínimo com Dijkstra

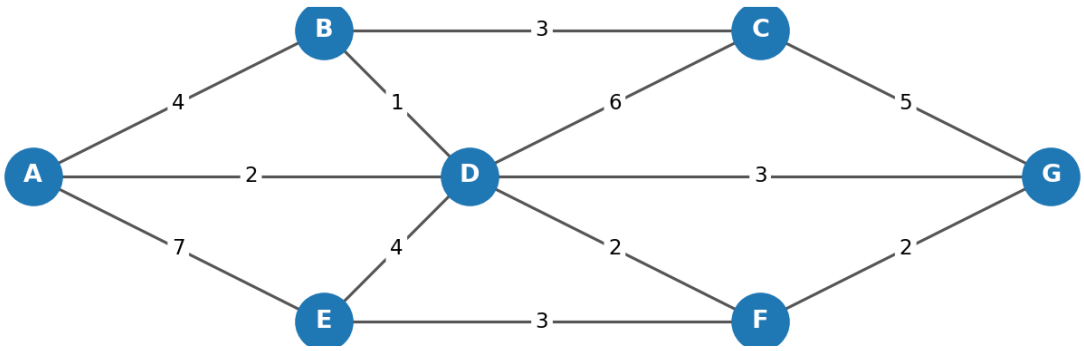
Trânsito entre pontos de uma cidade - relatório de testes

1. Problema e modelagem

Uma empresa de entregas precisa da menor rota entre dois pontos da cidade. A cidade é um grafo ponderado: vértices são locais, arestas são ruas de mão dupla e o peso é a distância em km. O grafo é guardado como **lista de adjacência** (dicionário ponto -> [(vizinho, km)]); cada rua é inserida nos dois sentidos, resultando em 24 entradas para 12 ruas.

Código	Descrição
A	Depósito
B	Cruzamento B
C	Cruzamento C
D	Cruzamento D
E	Cruzamento E
F	Cruzamento F
G	Cliente

Origem	Destino	Distância (km)
A	B	4
A	D	2
A	E	7
B	C	3
B	D	1
C	D	6
C	G	5
D	E	4
D	F	2
D	G	3
E	F	3
F	G	2



2. Algoritmo de Dijkstra

1. A distância da origem começa em 0 e a de todos os outros pontos em infinito.
2. Uma fila de prioridade (heapq) sempre retira o ponto com a menor distância ainda não fechada.

3. Para cada vizinho, se distância(atual) + peso for menor que a distância conhecida do vizinho, ela é atualizada e o atual vira o **predecessor** do vizinho.
4. Quando o destino sai da fila, sua distância é definitiva.
5. O caminho é reconstruído seguindo os predecessores do destino até a origem e invertendo a lista. Se o destino não tem predecessor, não existe caminho.

O algoritmo fica em graph.py, separado da interface (main.py) e do desenho (draw.py). Nenhuma função pronta de caminho mínimo foi usada.

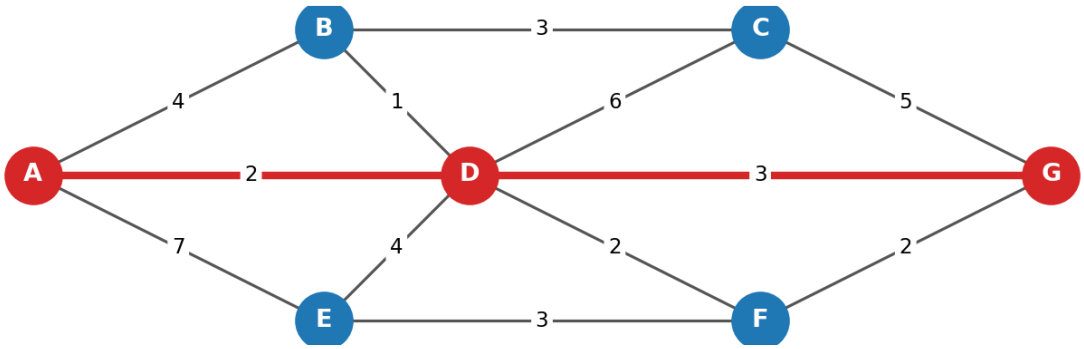
3. Testes

Cada caso foi calculado pelo Dijkstra e conferido por força bruta: todos os caminhos possíveis sem repetir pontos foram enumerados e o menor deles bate com o resultado. O teste 2 troca origem e destino do teste 1.

#	Origem	Destino	Caminho	km
1	A	G	A -> D -> G	5
2	G	A	G -> D -> A	5
3	A	C	A -> D -> B -> C	6
4	E	C	E -> D -> B -> C	8
5	E	G	E -> F -> G	5

Teste 1: A -> G = 5 km

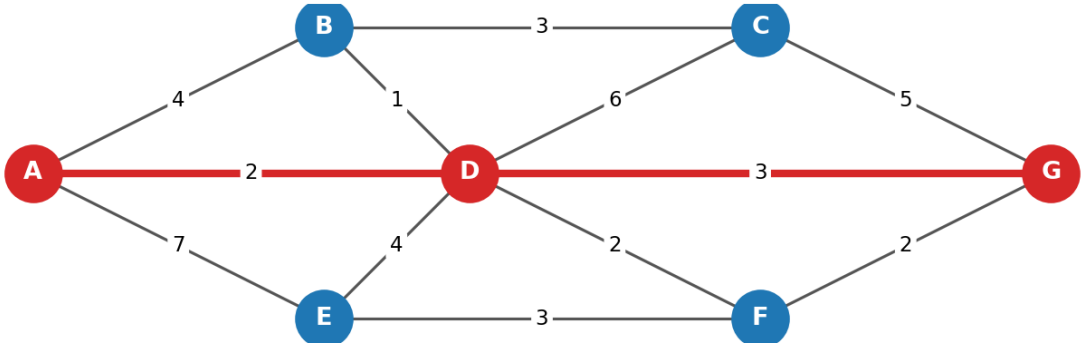
Caminho: **A -> D -> G**. 21 caminhos possíveis sem repetir pontos. Os 3 menores: A-D-G (5 km), A-D-F-G (6 km), A-B-D-G (8 km).



Teste 2: G -> A = 5 km

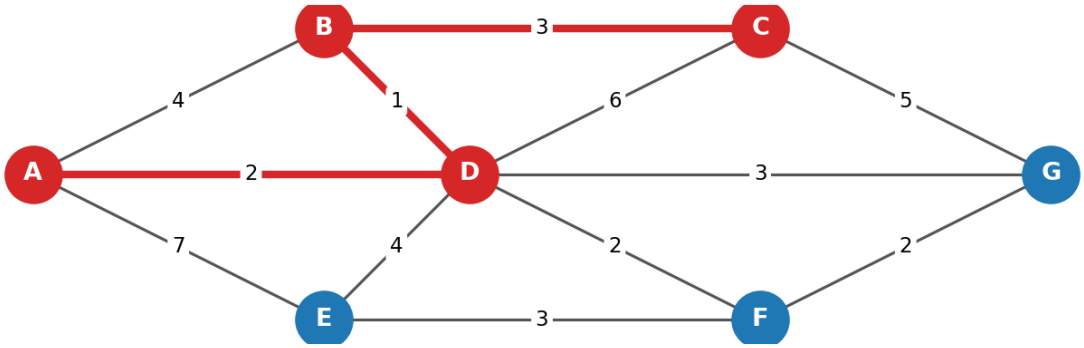
Caminho: **G -> D -> A**. 21 caminhos possíveis sem repetir pontos. Os 3 menores: G-D-A (5 km), G-F-D-A (6 km), G-D-B-A (8 km).

Mesmo caminho do teste 1 invertido e mesma distância: as ruas são de mão dupla.



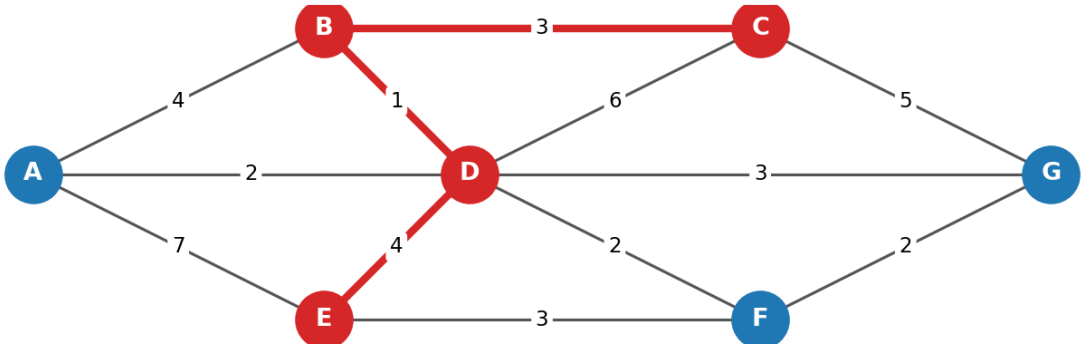
Teste 3: A -> C = 6 km

Caminho: **A -> D -> B -> C**. 20 caminhos possíveis sem repetir pontos. Os 3 menores: A-D-B-C (6 km), A-B-C (7 km), A-D-C (8 km).



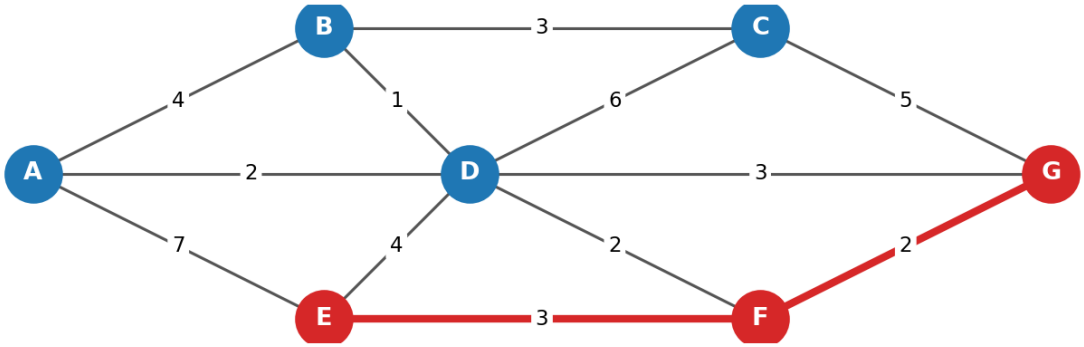
Teste 4: E -> C = 8 km

Caminho: **E -> D -> B -> C**. 21 caminhos possíveis sem repetir pontos. Os 3 menores: E-D-B-C (8 km), E-F-D-B-C (9 km), E-D-C (10 km).



Teste 5: E -> G = 5 km

Caminho: **E -> F -> G**. 20 caminhos possíveis sem repetir pontos. Os 3 menores: E-F-G (5 km), E-D-G (7 km), E-D-F-G (8 km).

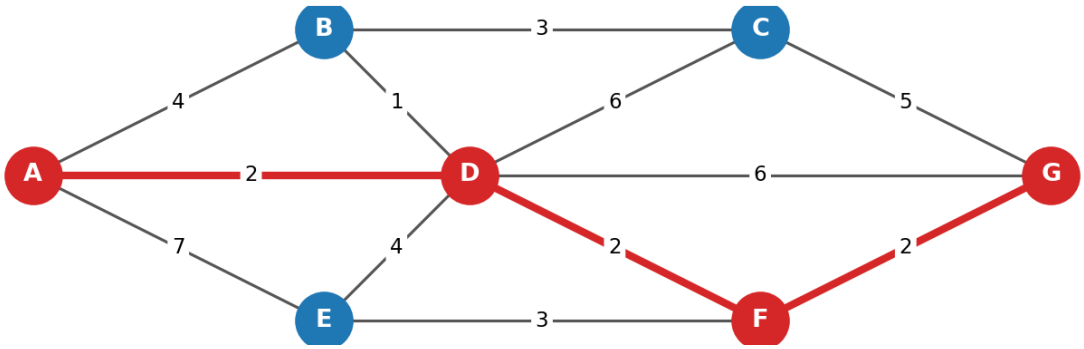


4. Desafio: alterar e desativar ruas

O Dijkstra só enxerga os pesos. Se uma rua do caminho mínimo fica mais longa ou fecha, outra soma pode ficar menor e a rota muda. Se a rua alterada não está no caminho, nada muda.

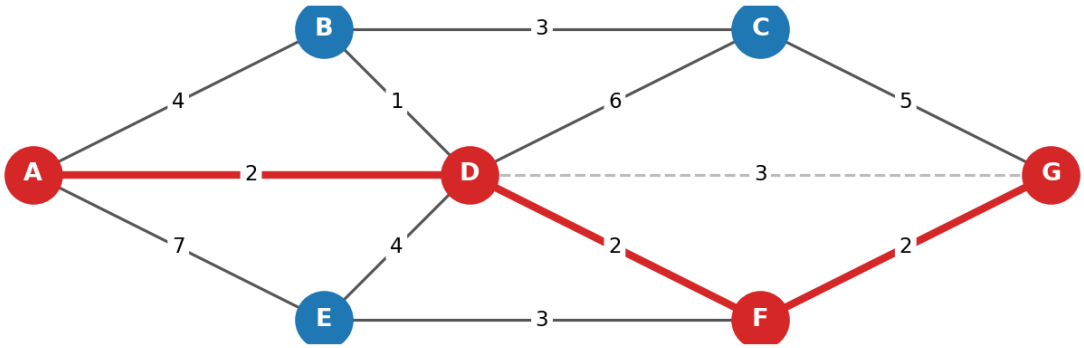
Trânsito: rua D-G passa de 3 para 6 km

Rota original: A -> D -> G (5 km). No caminho original, a rua D-G mudou de 3 para 6 km. Esse caminho agora custaria 8 km, então o Dijkstra escolheu A -> D -> F -> G (6 km).



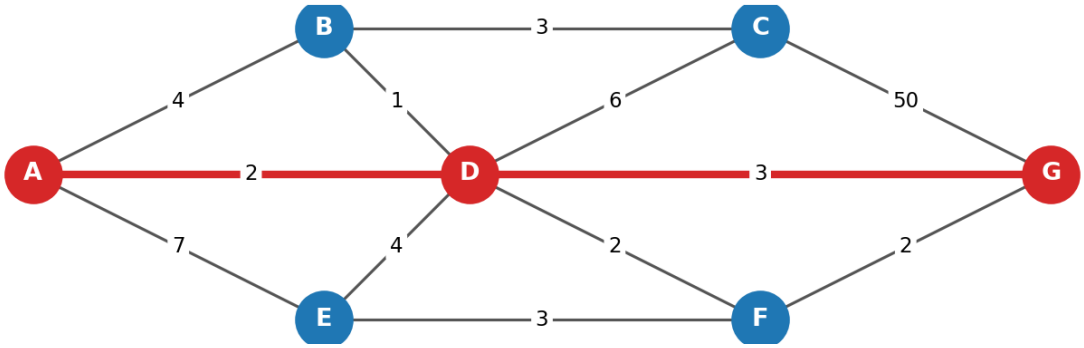
Obra: rua D-G desativada

Rota original: A -> D -> G (5 km). No caminho original, a rua D-G foi desativada. Esse caminho deixou de existir, então o Dijkstra escolheu A -> D -> F -> G (6 km).



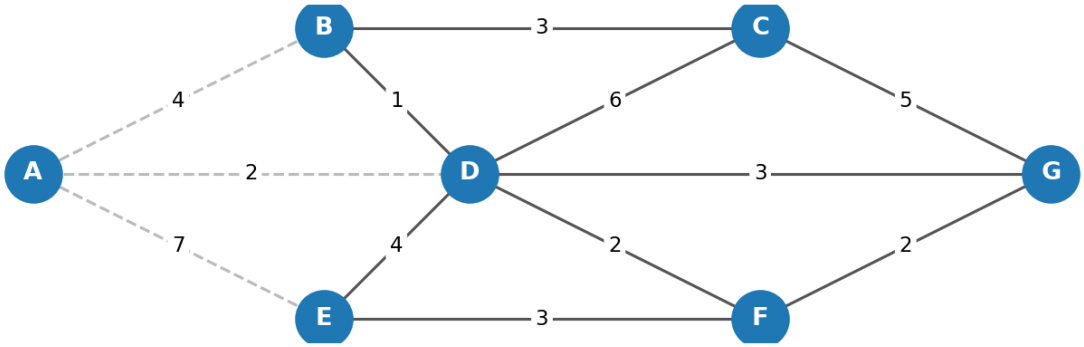
Rua fora da rota: C-G passa de 5 para 50 km

A rota não mudou: A -> D -> G continua sendo a menor (antes 5 km, agora 5 km).



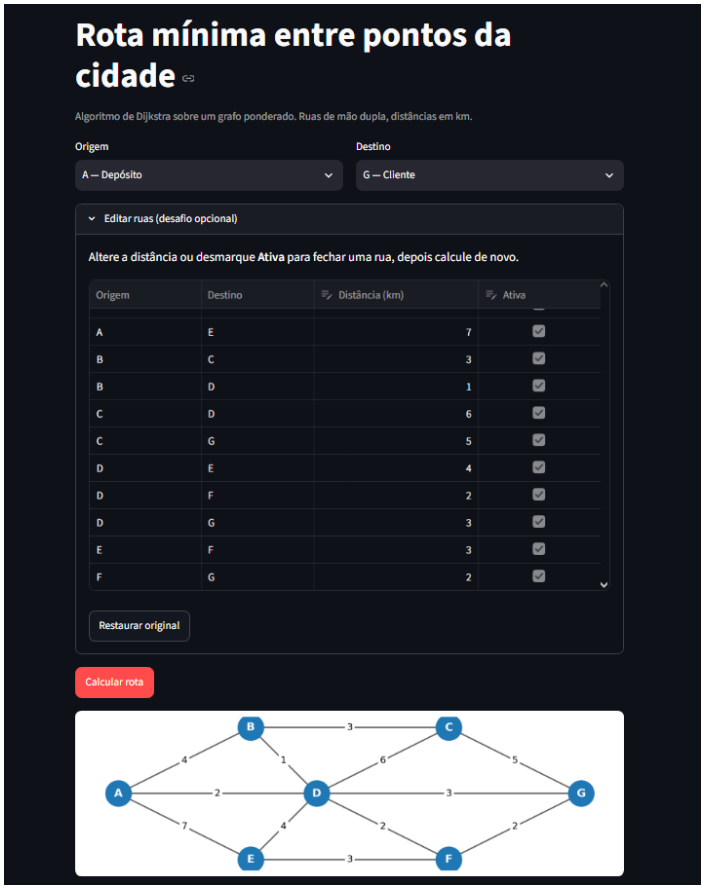
A isolado: ruas A-B, A-D e A-E desativadas

Não existe caminho entre A e G: a aplicação mostra uma mensagem de erro.

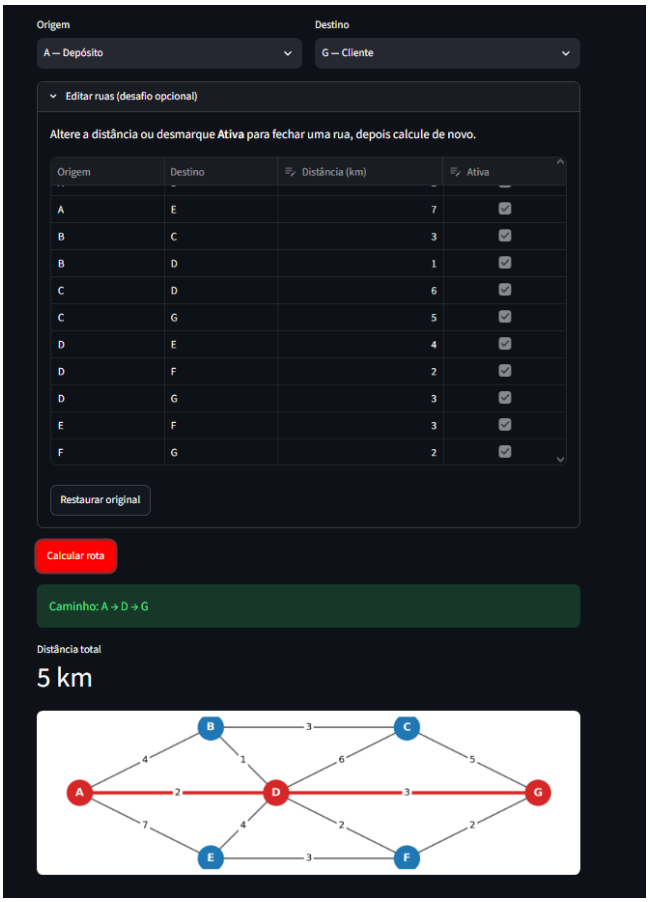


5. Capturas de tela da interface

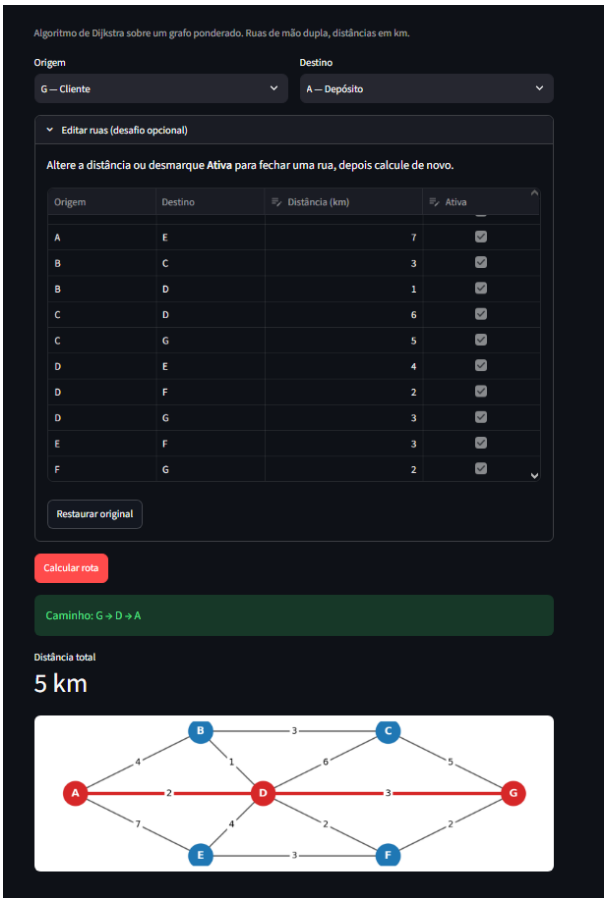
01-interface



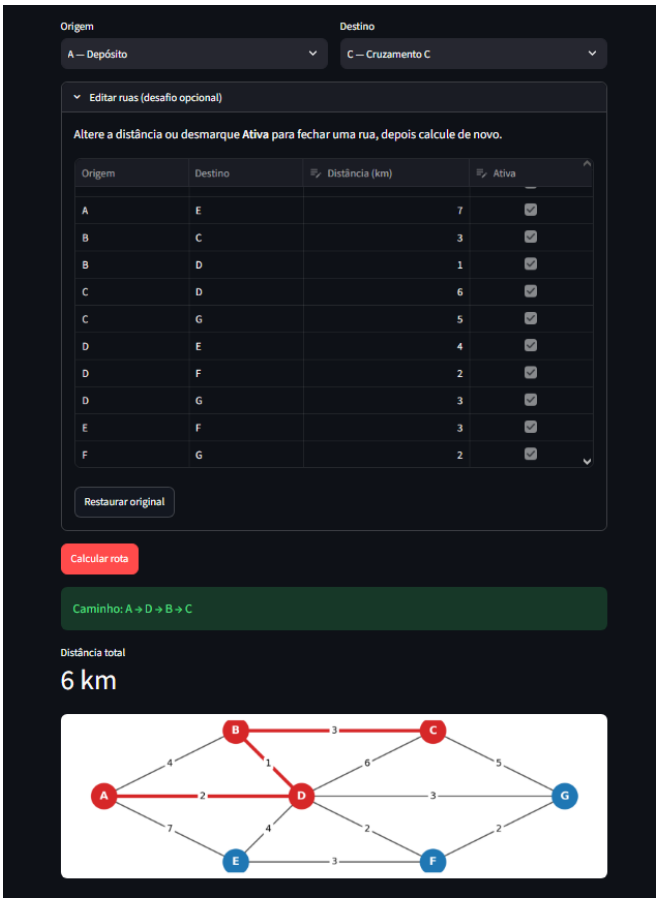
02-teste-A-G



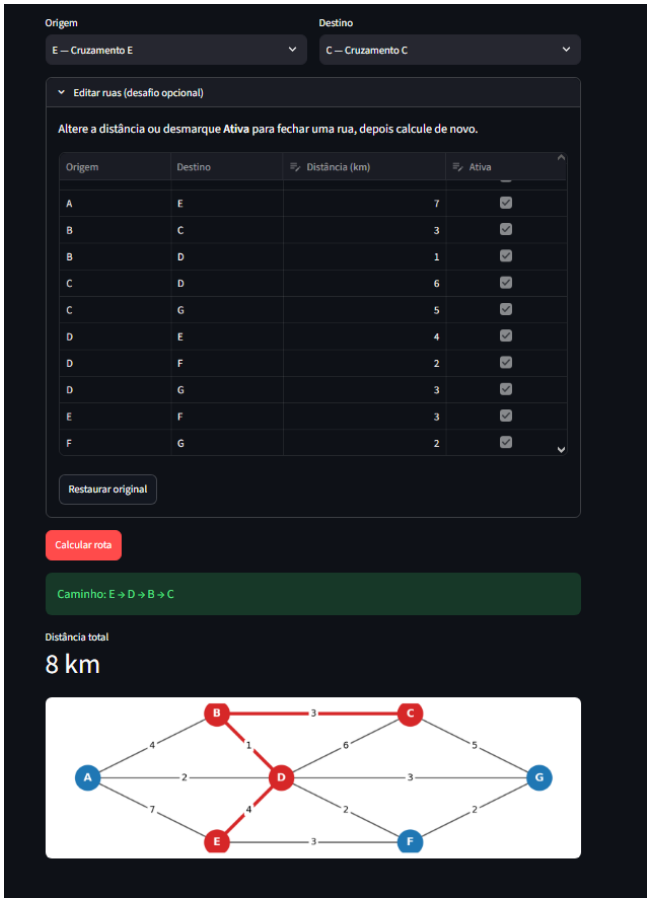
03-teste-G-A-trocado



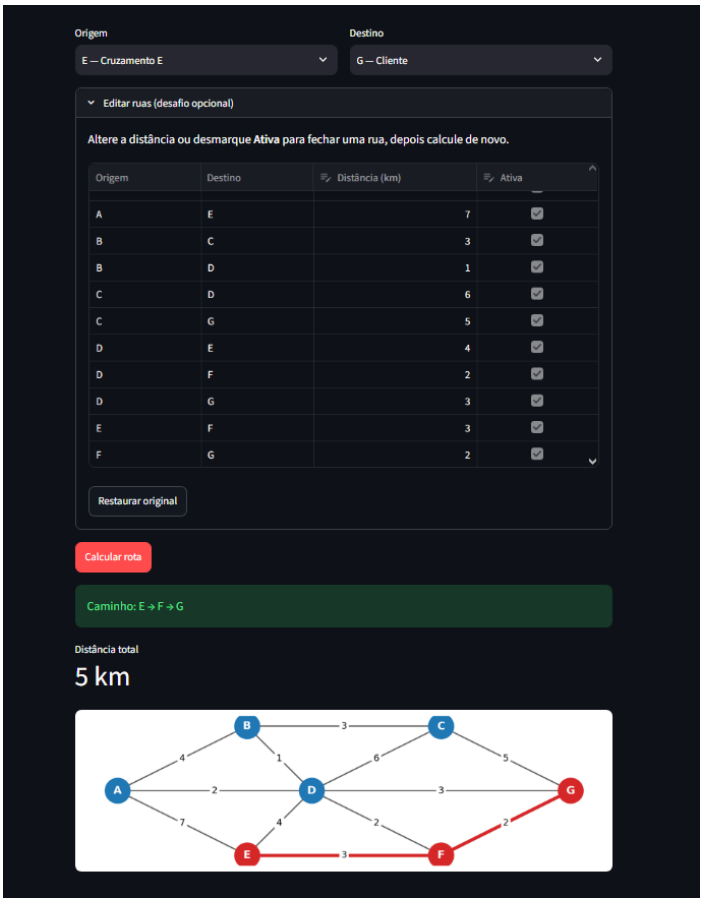
04-teste-A-C



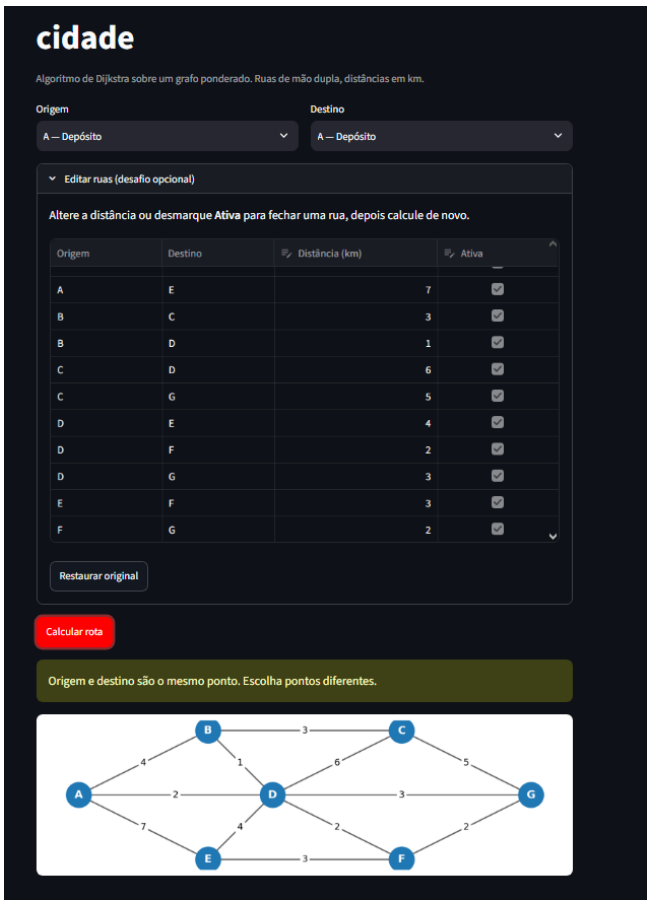
05-teste-E-C



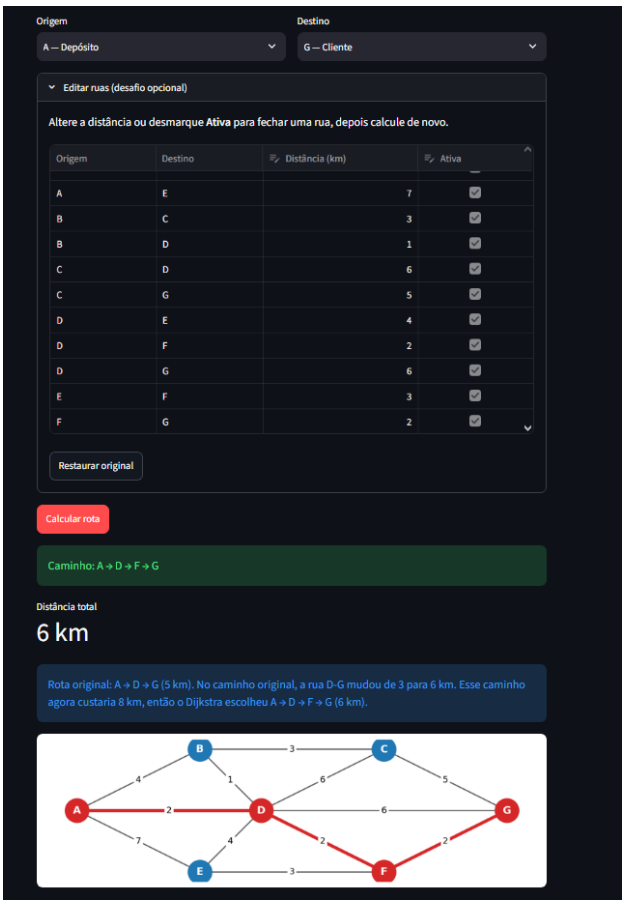
06-teste-E-G



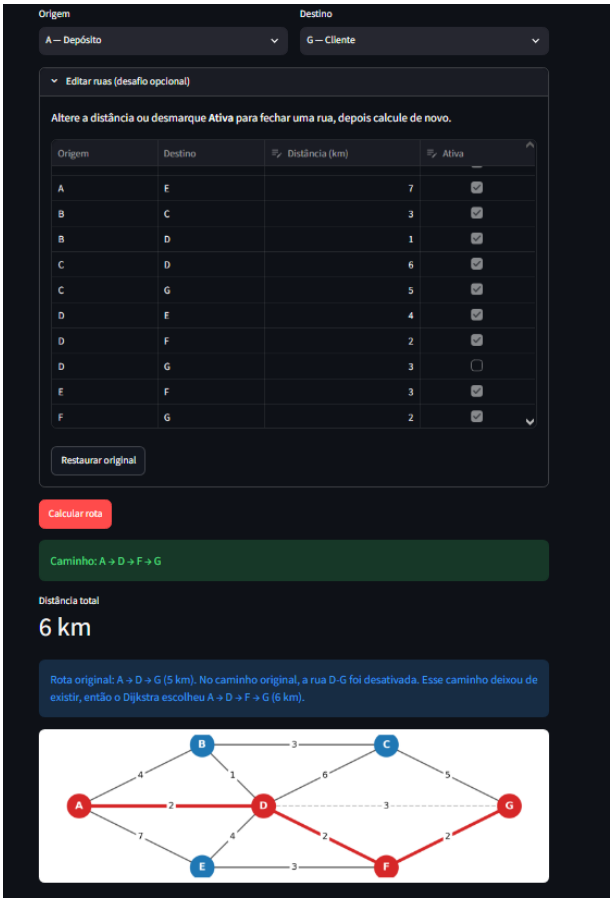
07-mesmo-ponto



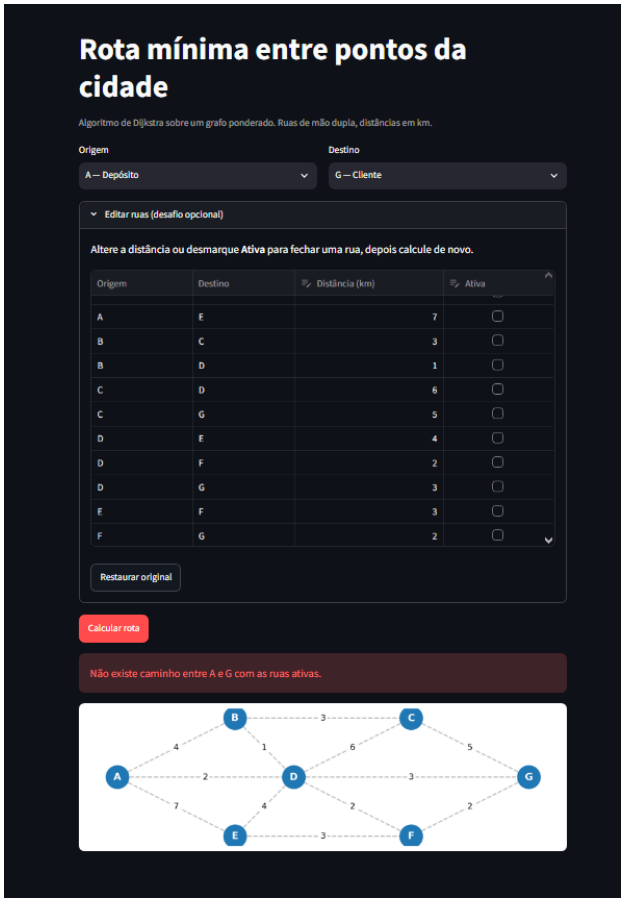
08-desafio-DG-6km



09-desafio-DG-desativada



10-sem-caminho



6. Como executar

pip install -r requirements.txt
python main.py (abre em http://localhost:8501)
python test_graph.py (testes automáticos)
python report.py (gera este relatório)